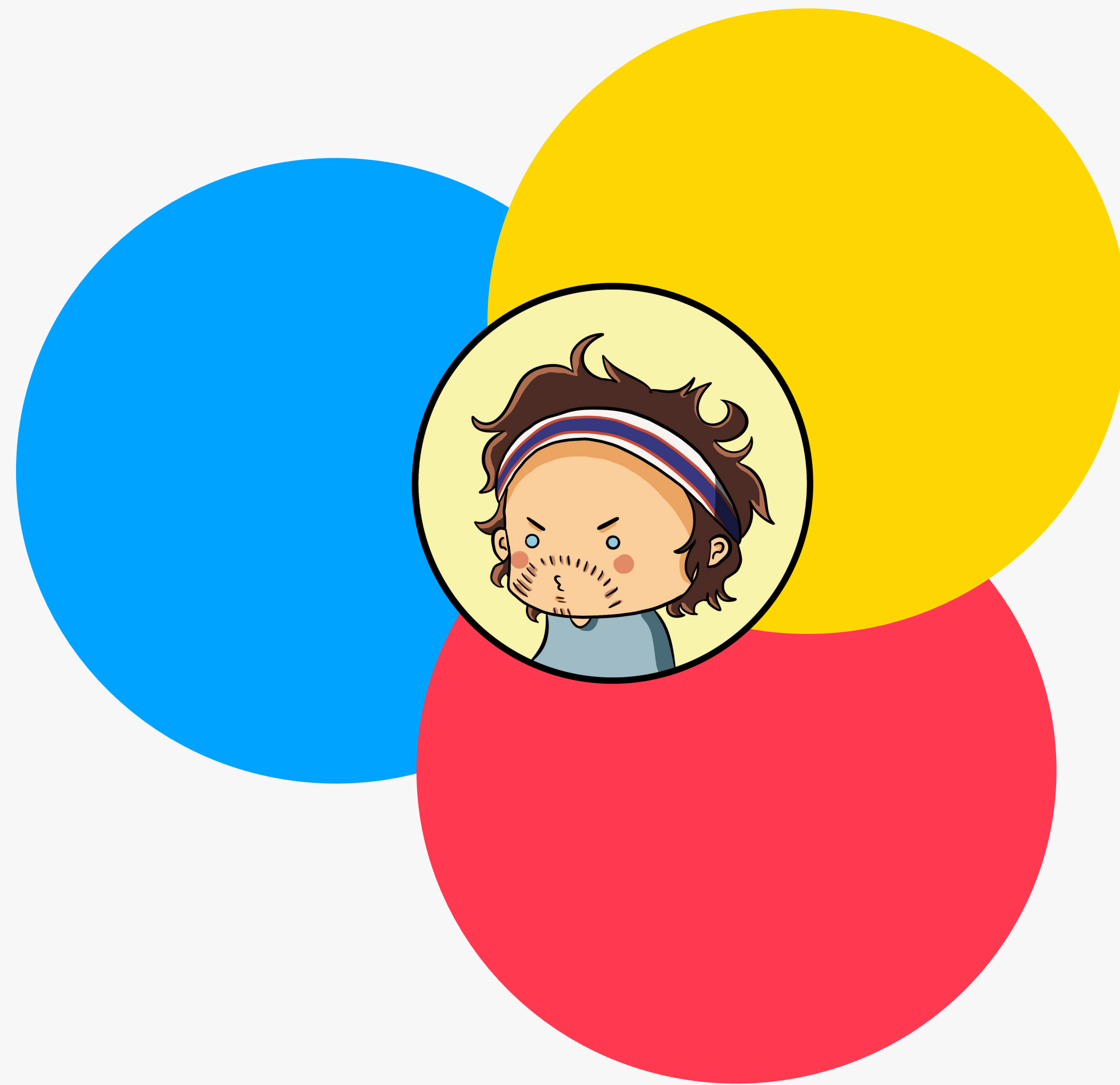




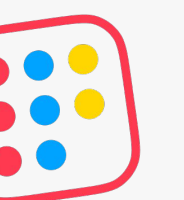
Git-for-everything and the self-driving data stack

Jacopo Tagliabue
HPTS, October 2026

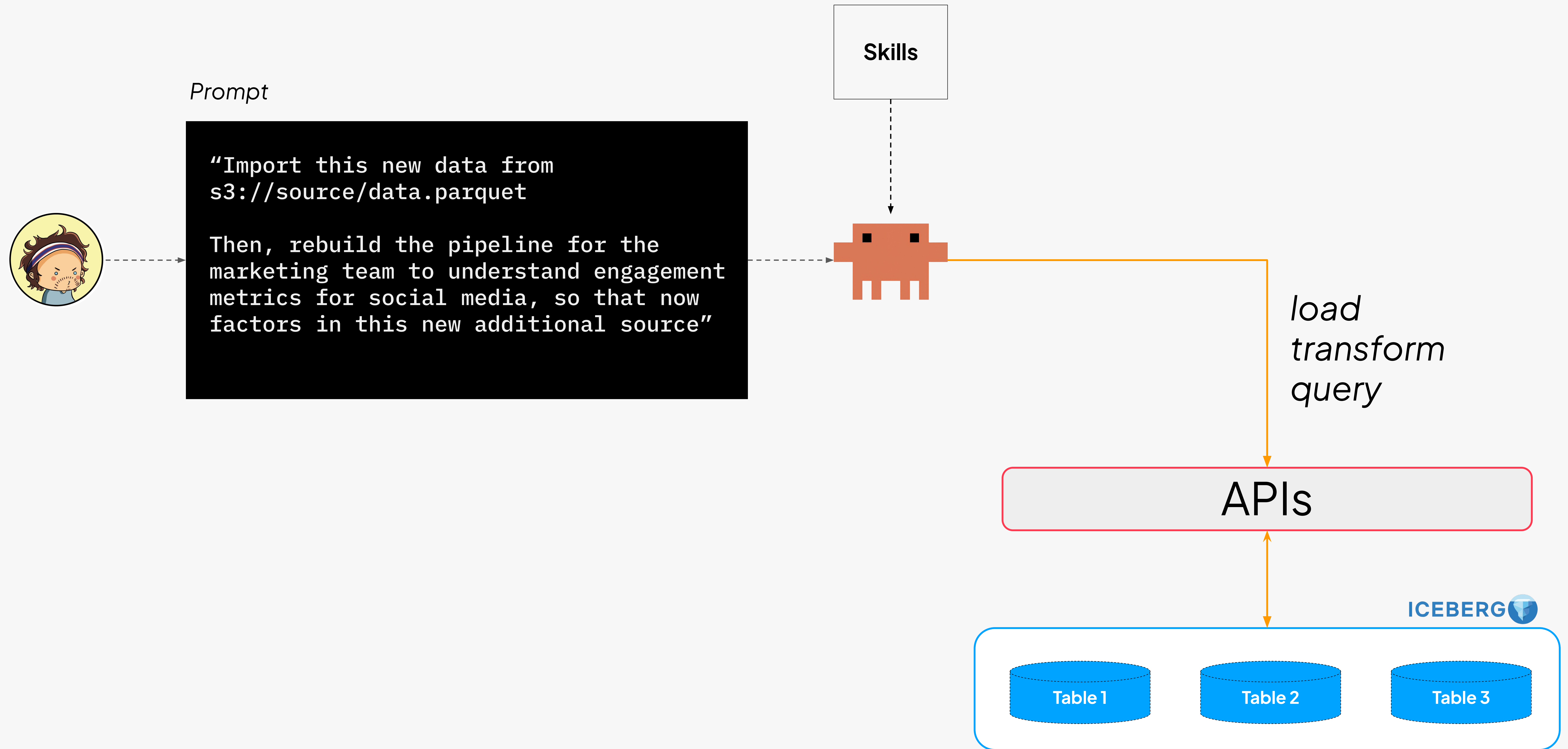


Ciao!

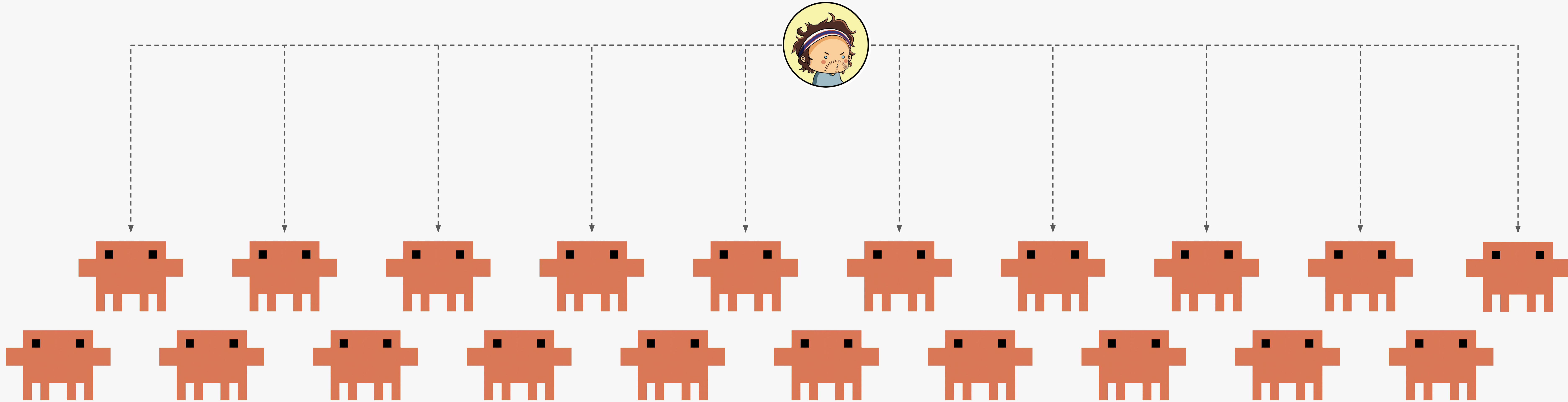
- | Co-founder at **Bauplan**, OLAP infrastructure for agents.
Backed by Index, IE, SPC, Wes McKinney, Spencer Kimball, Chris Ré, etc.
- | Previous episodes: CTO of Tooso, teaching MLSys at NYU.
Led AI at TSX:CVO after acquisition.
- | 10 years up and down the stack in R&D, product, open source.
60+ papers (ICML, KDD, VLDB, NAACL, SIGIR...), >3k stars; 100M+ downloads.



Agentic OLAP today



Tomorrow?



1. Concurrent ops
2. Conflict management
3. Auditability

Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First

Shu Liu, Soujanya Ponnappalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu
Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia
Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, Aditya G. Parameswaran
University of California, Berkeley

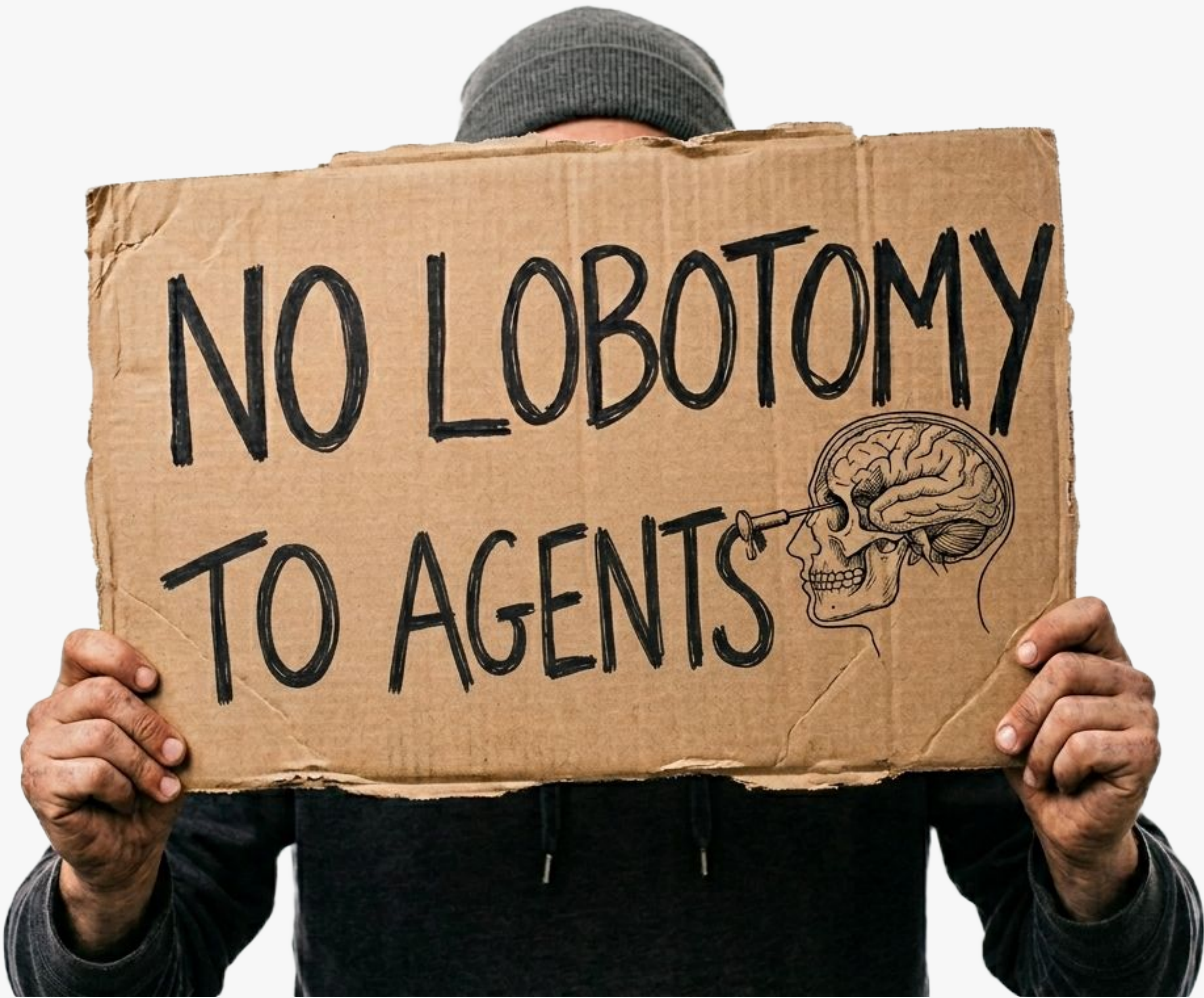
Abstract

Large Language Model (LLM) agents, acting on their users' behalf to manipulate and analyze data, are likely to become the dominant workload for data systems in the future. When working with data, agents employ a high-throughput process of exploration and solution formulation for the given task, one we call *agentic speculation*. The sheer volume and inefficiencies of agentic speculation can pose

of LLM agents tasked with finding reasons for why profits in coffee bean sales in Berkeley was low this year relative to last. Since they are not limited by human cognitive bandwidth and response times, an army of agents could employ an enormous volume of queries to data systems, far more than any human could—all for a single task. Many of these queries are likely wasteful, and are simply providing the agents grounding. As another example, if an LLM agent is tasked with identifying a new cream for a delayed flight, it could

Teams trade off autonomy for stability

*Teams deliberately constrain autonomy for production stability. **Only 1 case uses unconstrained exploration, exclusively in sandboxed environments with rigorous CI/CD verification.***



04123v3 [cs.CY] 3 Feb 2026

Measuring Agents in Production

Melissa Z. Pan^{1*} Negar Arabzadeh^{1*} Riccardo Cogo² Yuxuan Zhu³ Alexander Xiong¹ Lakshya A Agrawal¹
Huanzhi Mao¹ Emma Shen¹ Sid Pallerla¹ Liana Patel⁴ Shu Liu¹ Tianneng Shi¹ Xiaoyuan Liu¹
Jared Quincy Davis⁴ Emmanuele Lacavalla² Alessandro Basile² Shuyi Yang² Paul Castro⁵ Daniel Kang³
Koushik Sen¹ Dawn Song¹ Joseph E. Gonzalez¹ Ion Stoica¹ Matei Zaharia^{1*} Marquita Ellis^{5*}
¹UC Berkeley ²Intesa Sanpaolo ³UIUC ⁴Stanford University ⁵IBM Research

Abstract

LLM-based agents already operate in production across many industries, yet we lack an understanding of what technical methods make deployments successful. We present the first systematic study of **Measuring Agents in Production**, MAP, using first-hand data from agent developers. We conducted 20 case studies via in-depth interviews and surveyed 306 practitioners across 26 domains. We investigate why organizations build agents, how they build them, how they evaluate them, and their top development challenges. Our study finds that production agents are built using simple, controllable approaches: 68% execute at most 10 steps before human intervention, 70% rely on prompting off-the-shelf models instead of weight tuning, and 74% depend primarily on human evaluation. Reliability (consistent correct behavior over time) remains the top development challenge, which practitioners currently address through systems-

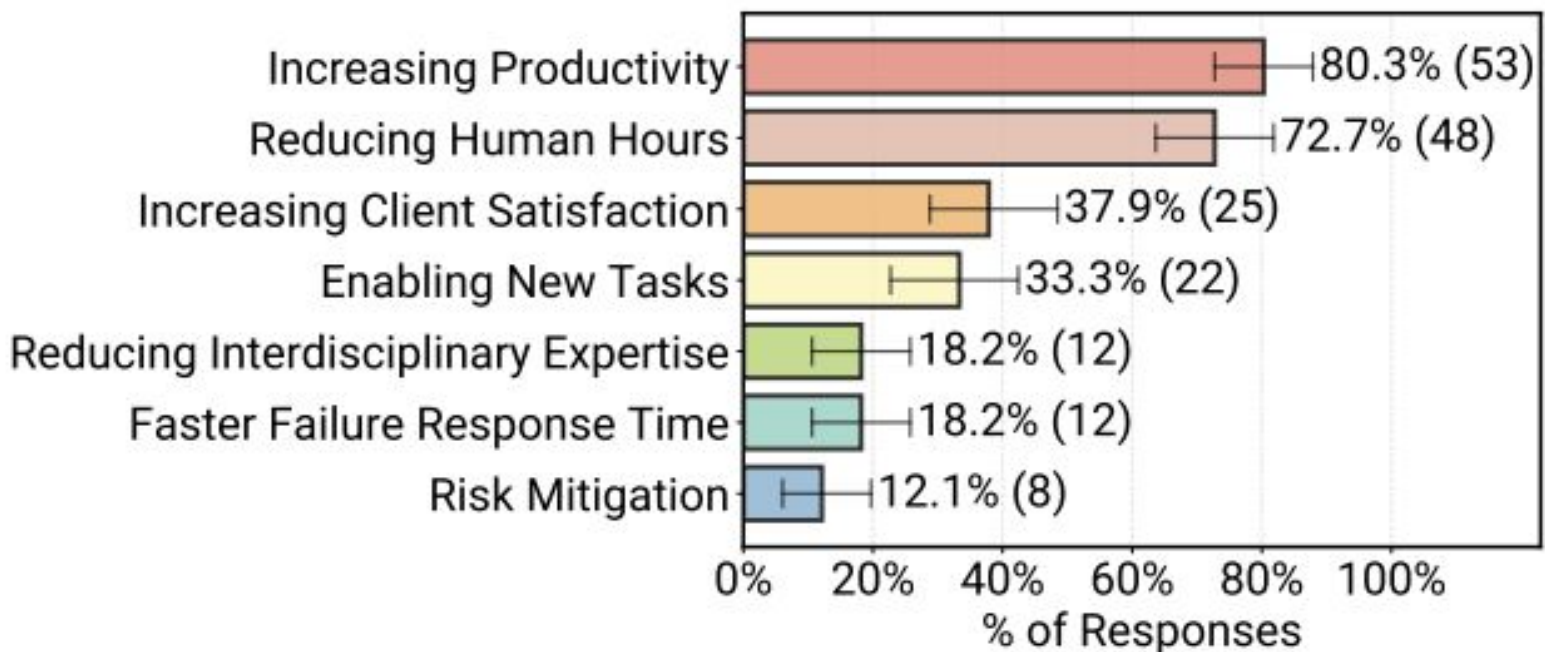
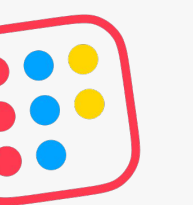


Figure 1. Reasons practitioners build and deploy AI agents (N=66). The question is multi-select, so proportions do not sum to 1. Error bars indicate 95th percentile intervals estimated from 1,000 bootstrap samples with replacement.

Despite widespread excitement about agents, studies show agent deployments often fail or underdeliver (Xue et al., 2025; Reuters Staff, 2025; Shome et al., 2025). The stark contrast between the potential of agents and their failures raises the fundamental question of what enables successful agent deployment. The field can only advance collectively

This is the wrong
way to scale



How do we build a self-driving data stack?



- 1. A familiar mental model**
- 2. Some honest systems work**

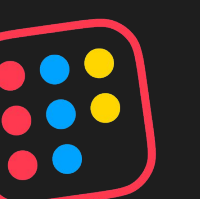


Software

Data



TRIAL AND ERROR



Software

Data

Modern eng principles & practices

Modularity / separation of concerns

Incremental development

External dependencies, libraries, frameworks

Debugger

Change management

Testing (unit tests, integration tests)

Continuous integration / continuous delivery

... SQL makes it virtually impossible to apply these.

2019!



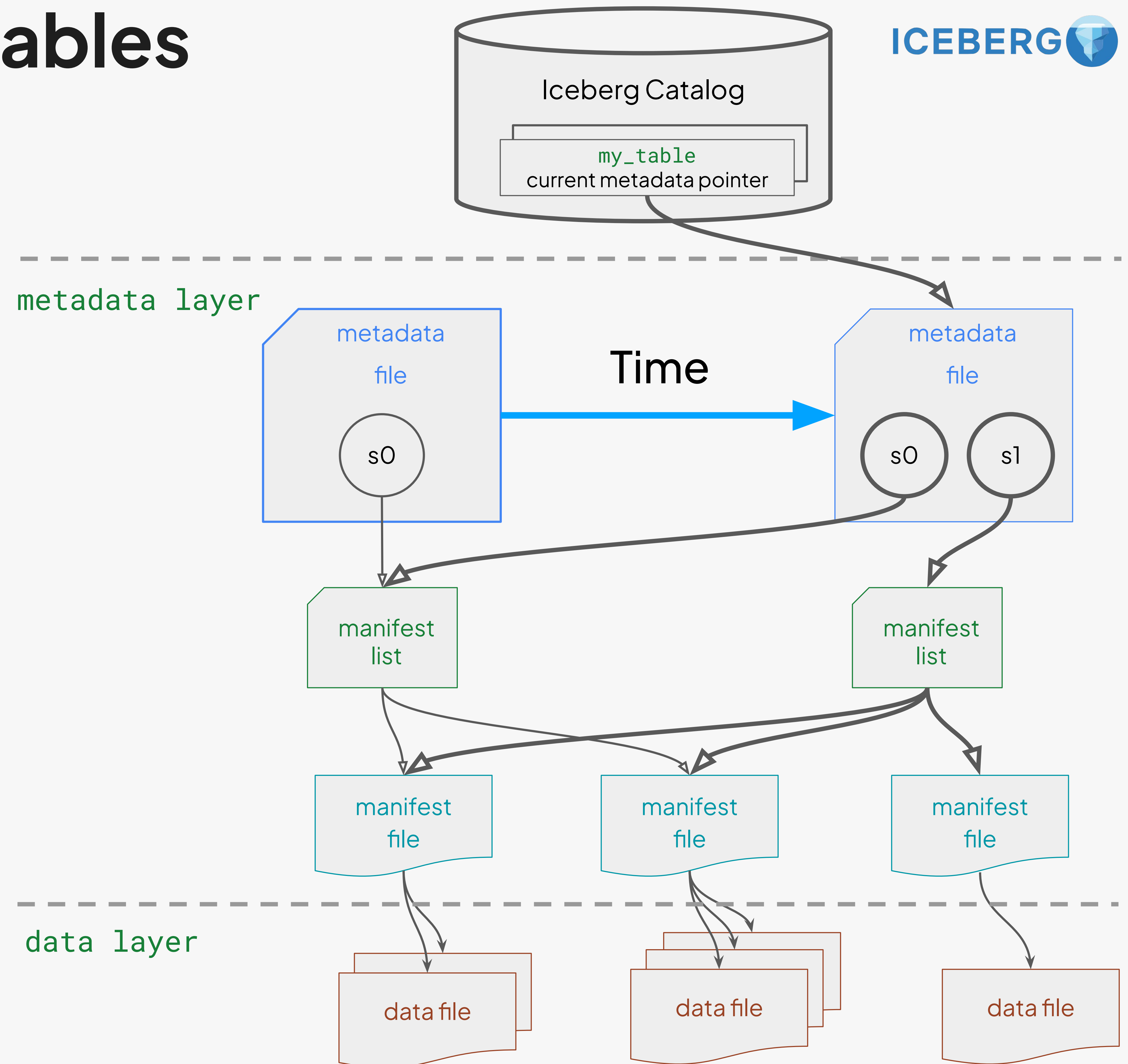
Git, but for data



From files to (Iceberg) tables

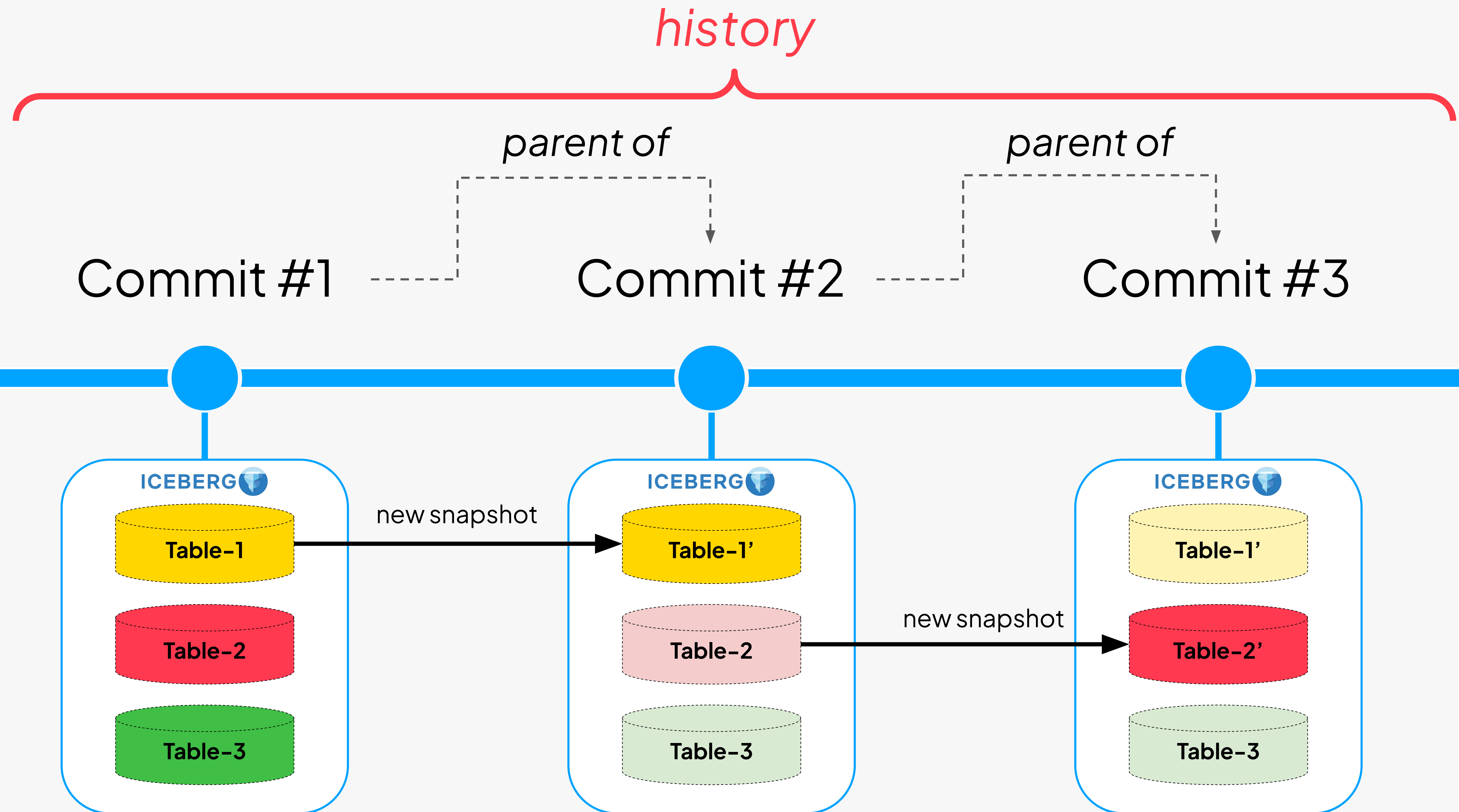
Apache Iceberg brings database-like tables to object storage:

- **Table atomicity**
ACID *per table* – no conflicting or partial writes.
- **Optimistic locking**
Tables change through snapshot evolution, governed by a database which makes sure no two writes can update the same one.



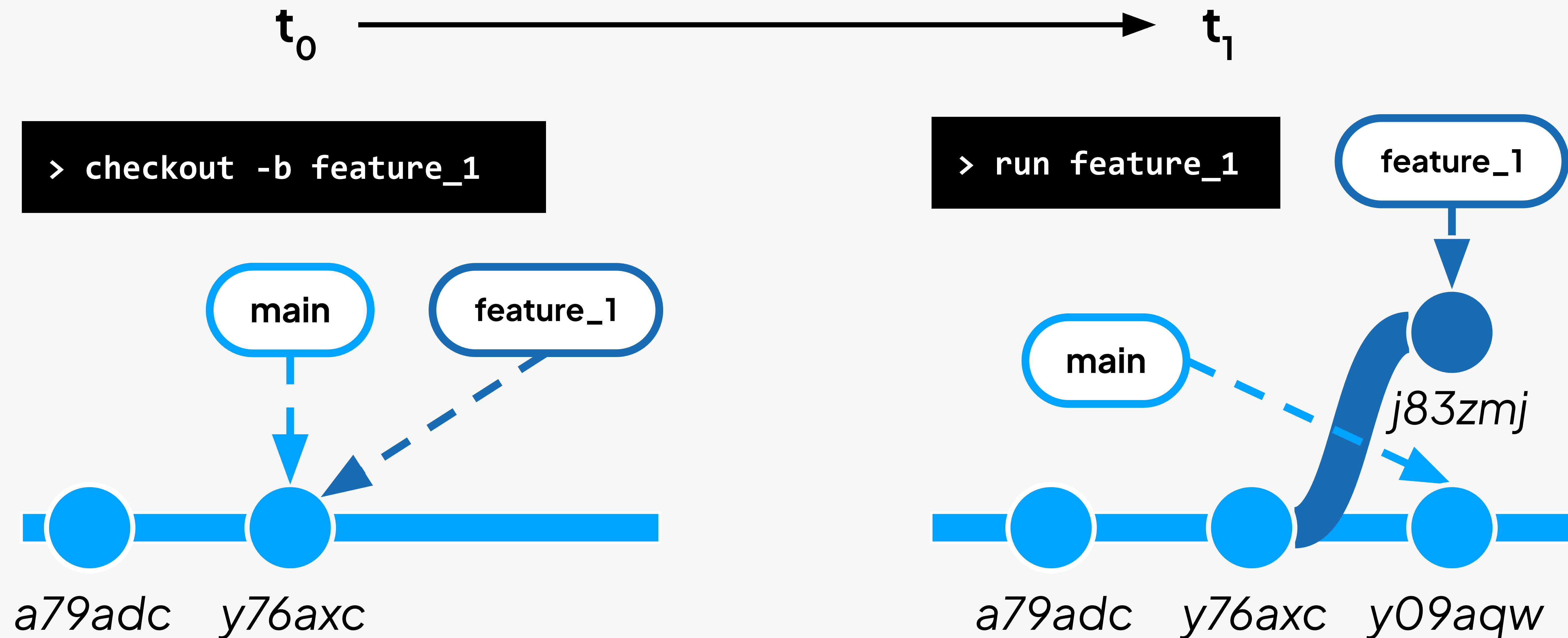
From tables to commits

- **Commit:** a map $\langle \text{table}, \text{snapshot} \rangle$ at t (for every write)



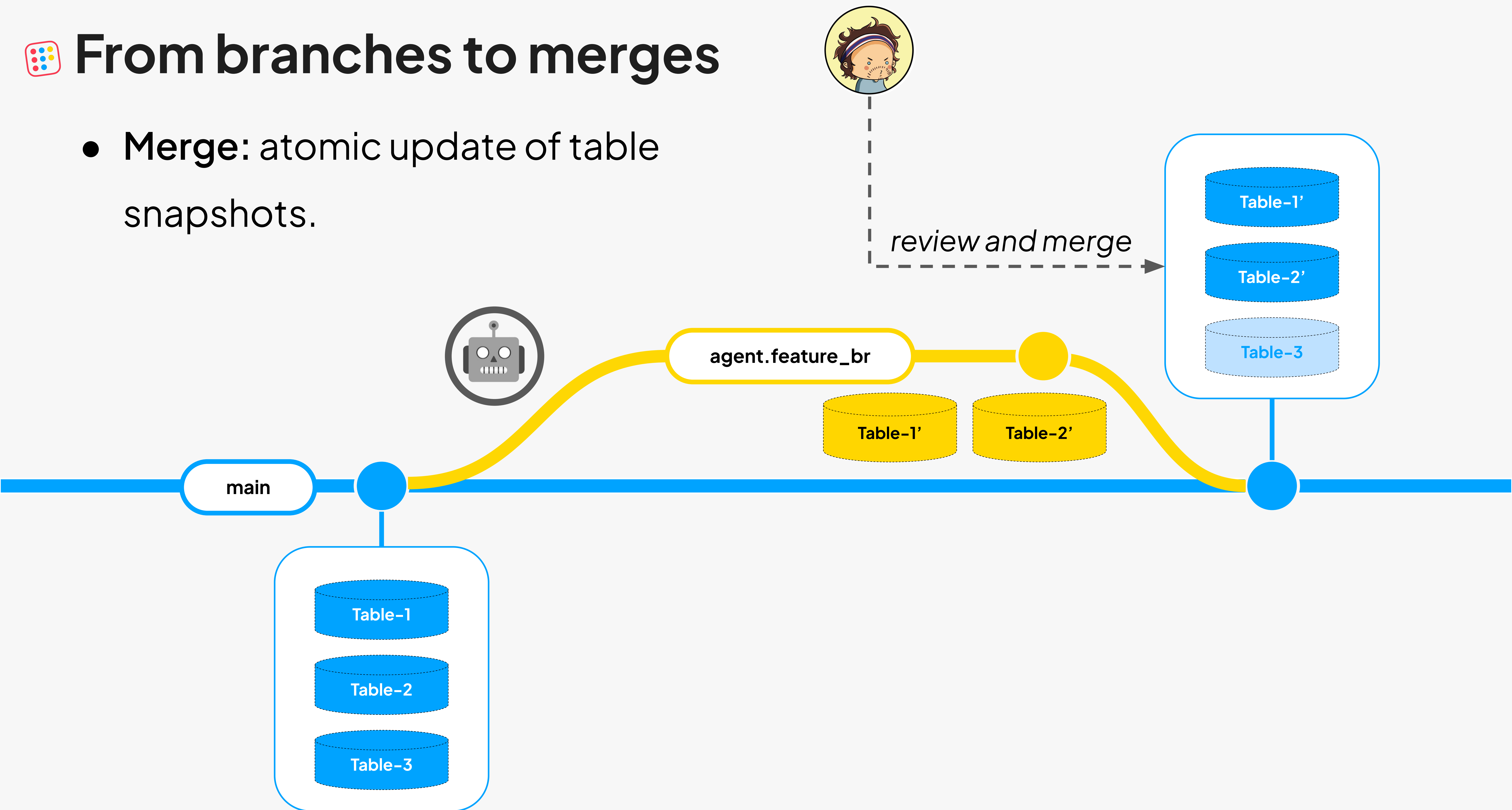
From commits to branches

- **Branch:** movable reference to the HEAD of a *history*.
- **Time travel:** read ops accept *commits* as argument.

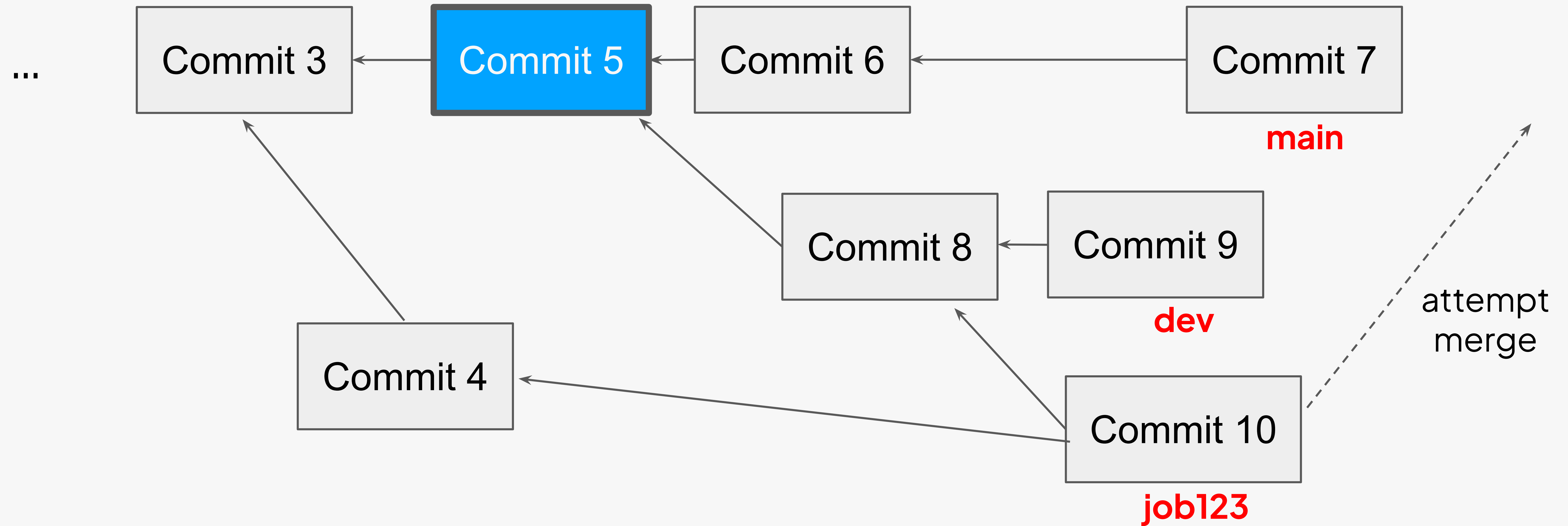


From branches to merges

- Merge: atomic update of table snapshots.

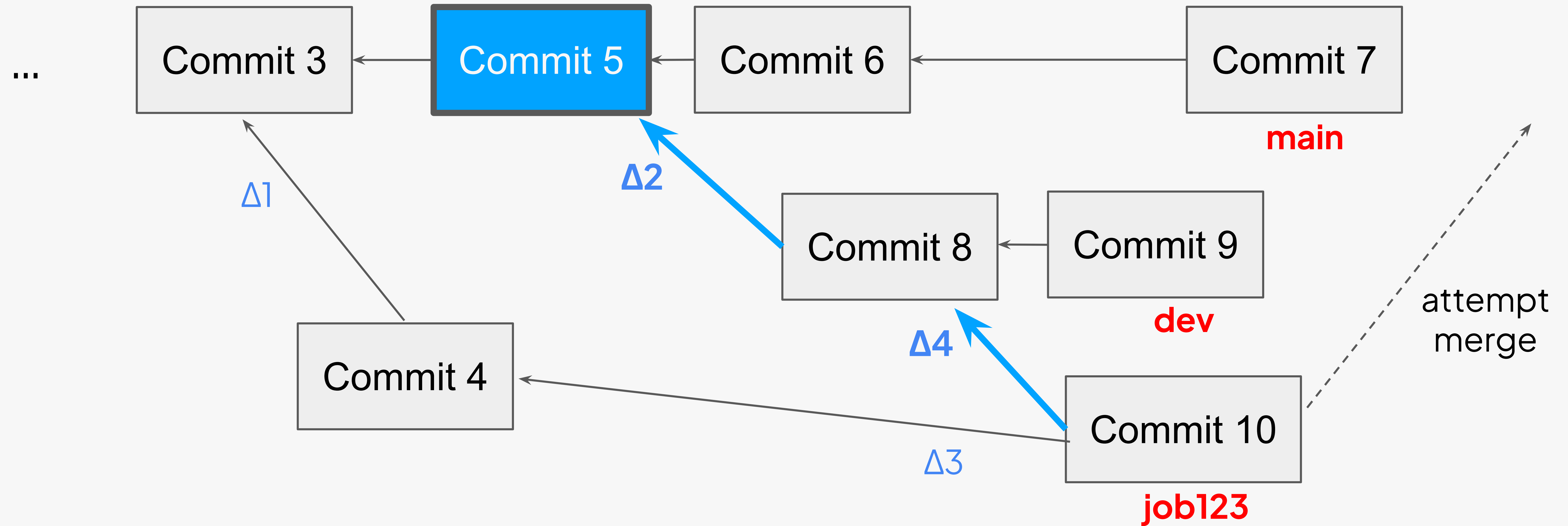


From branches to merges



Step 1: find the common ancestor of commits 7 and 10

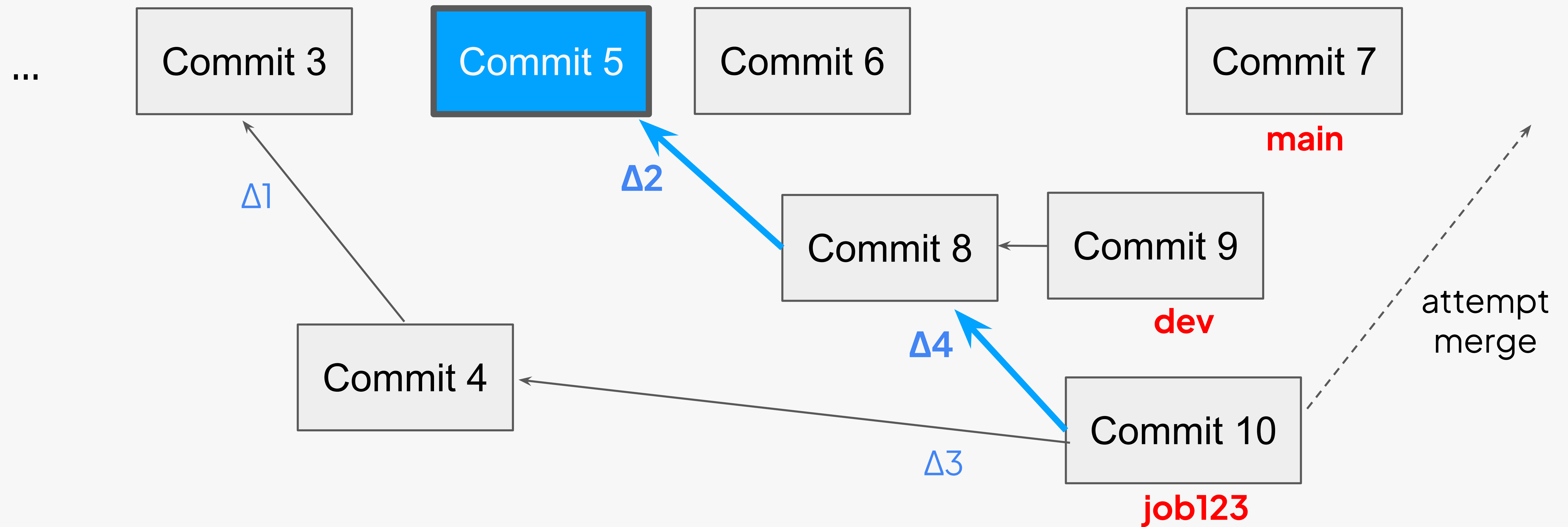
From branches to merges



Step 1: find the common ancestor of commits 7 and 10

Step 2: $\text{diff}(\text{commit5}, \text{commit10}) = \Delta 2 + \Delta 4$

From branches to merges



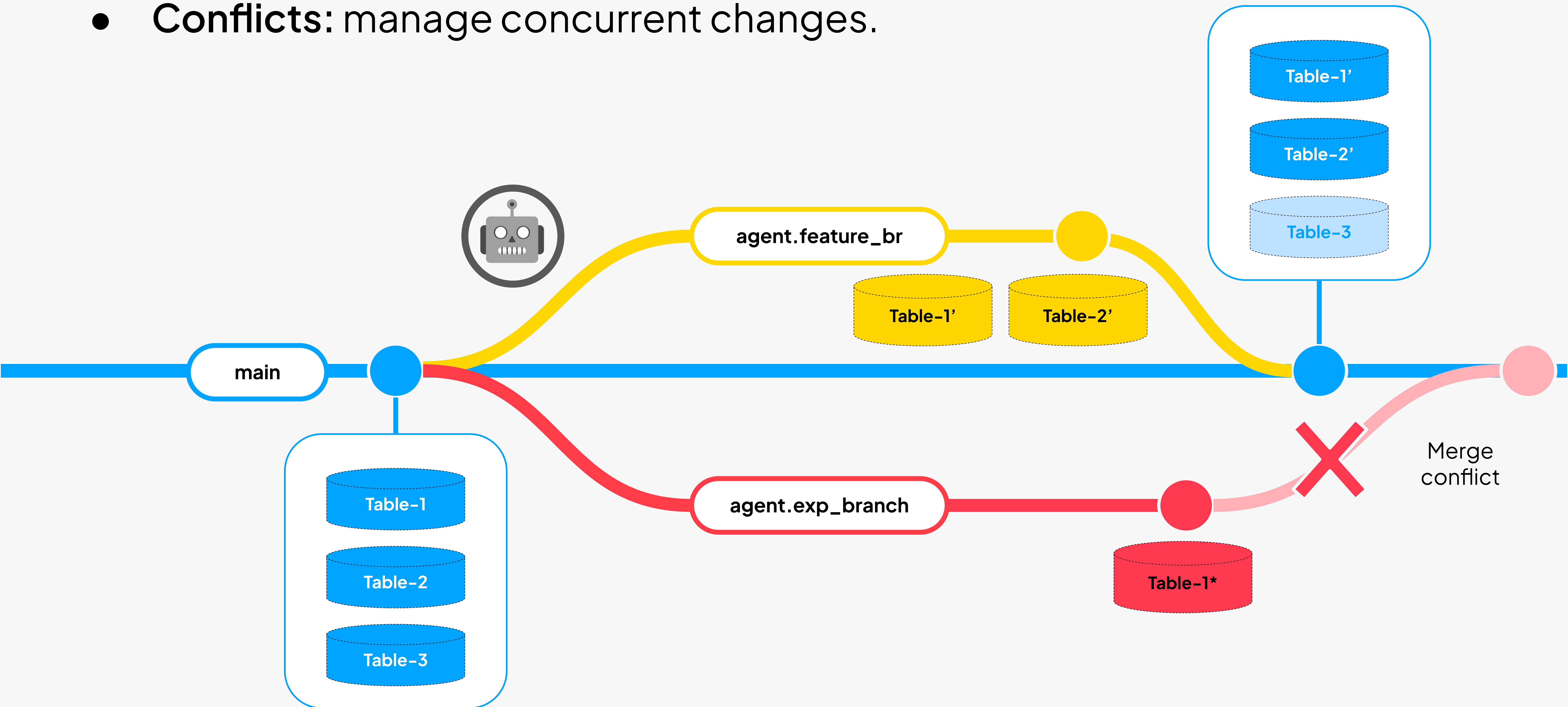
Step 1: find the common ancestor of commits 7 and 10

Step 2: $\text{diff}(\text{commit5}, \text{commit10}) = \Delta 2 + \Delta 4$

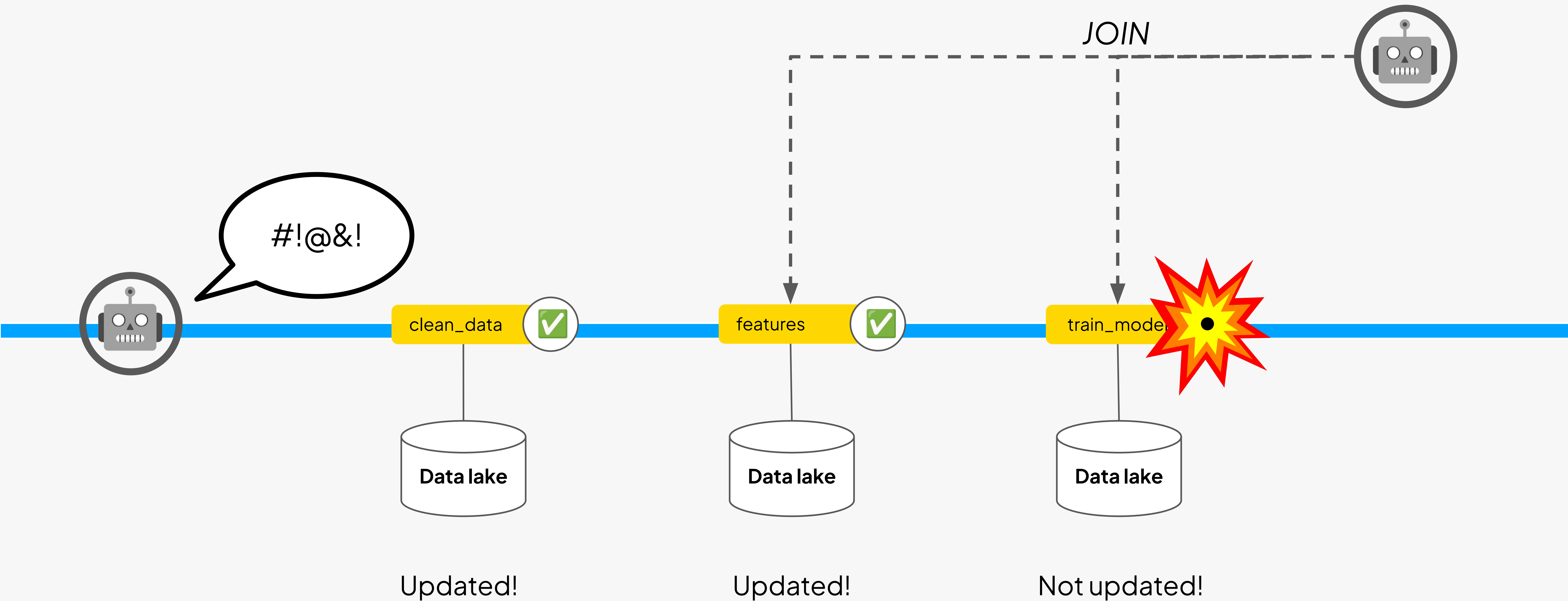
Step 3: apply patch to destination

From branches to merges

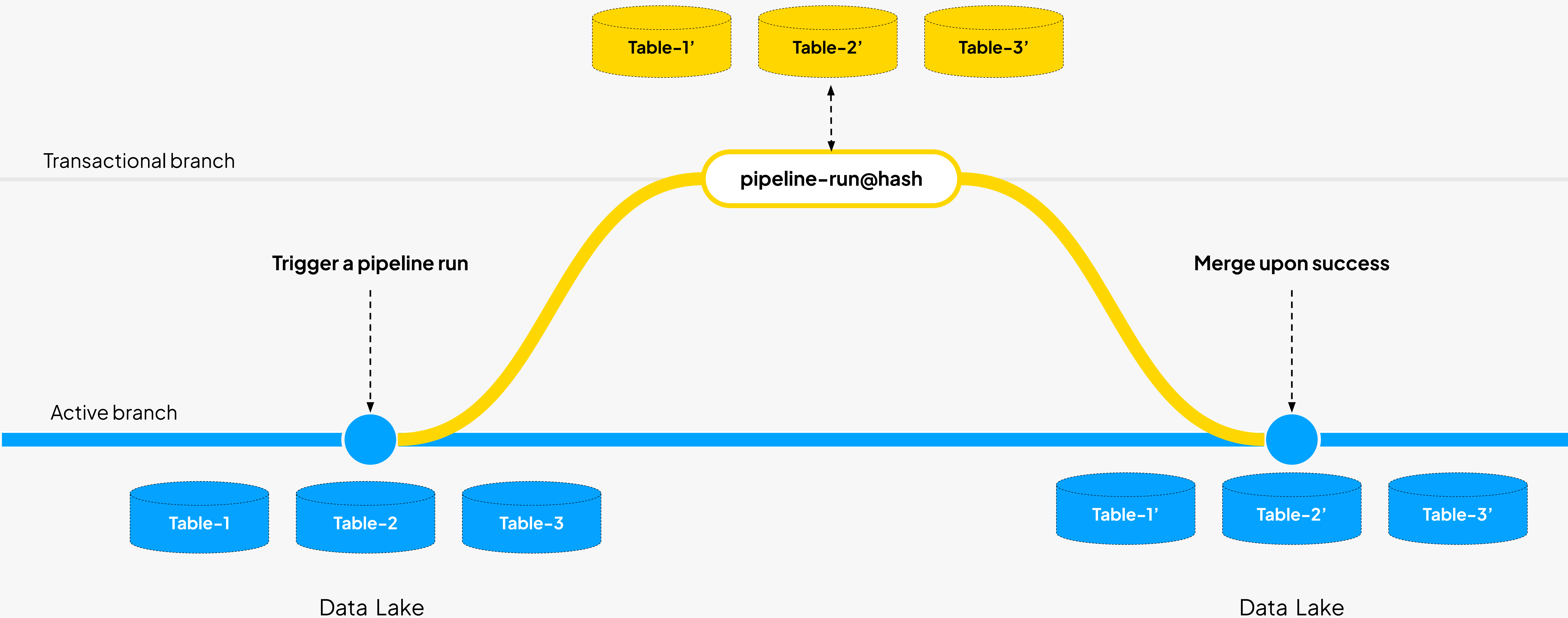
- **Conflicts:** manage concurrent changes.



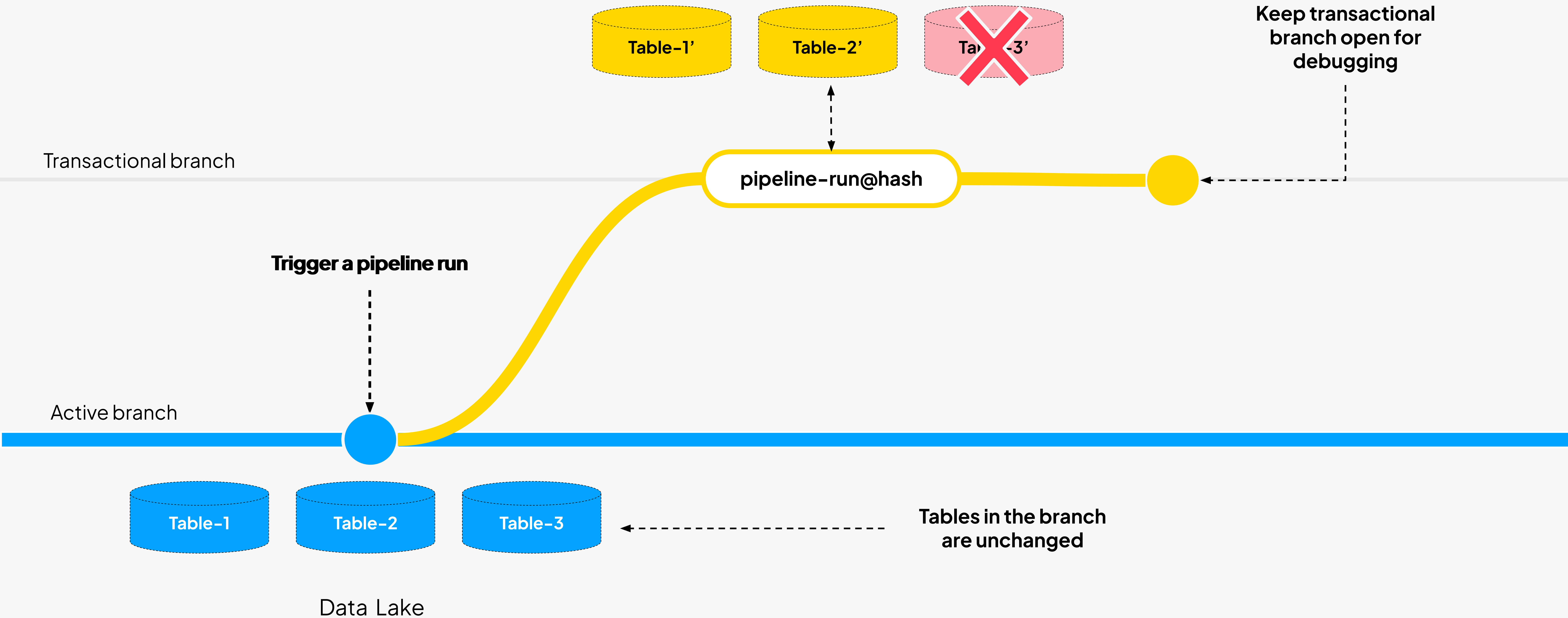
The curse of half-written pipelines



From merges to transactions: success

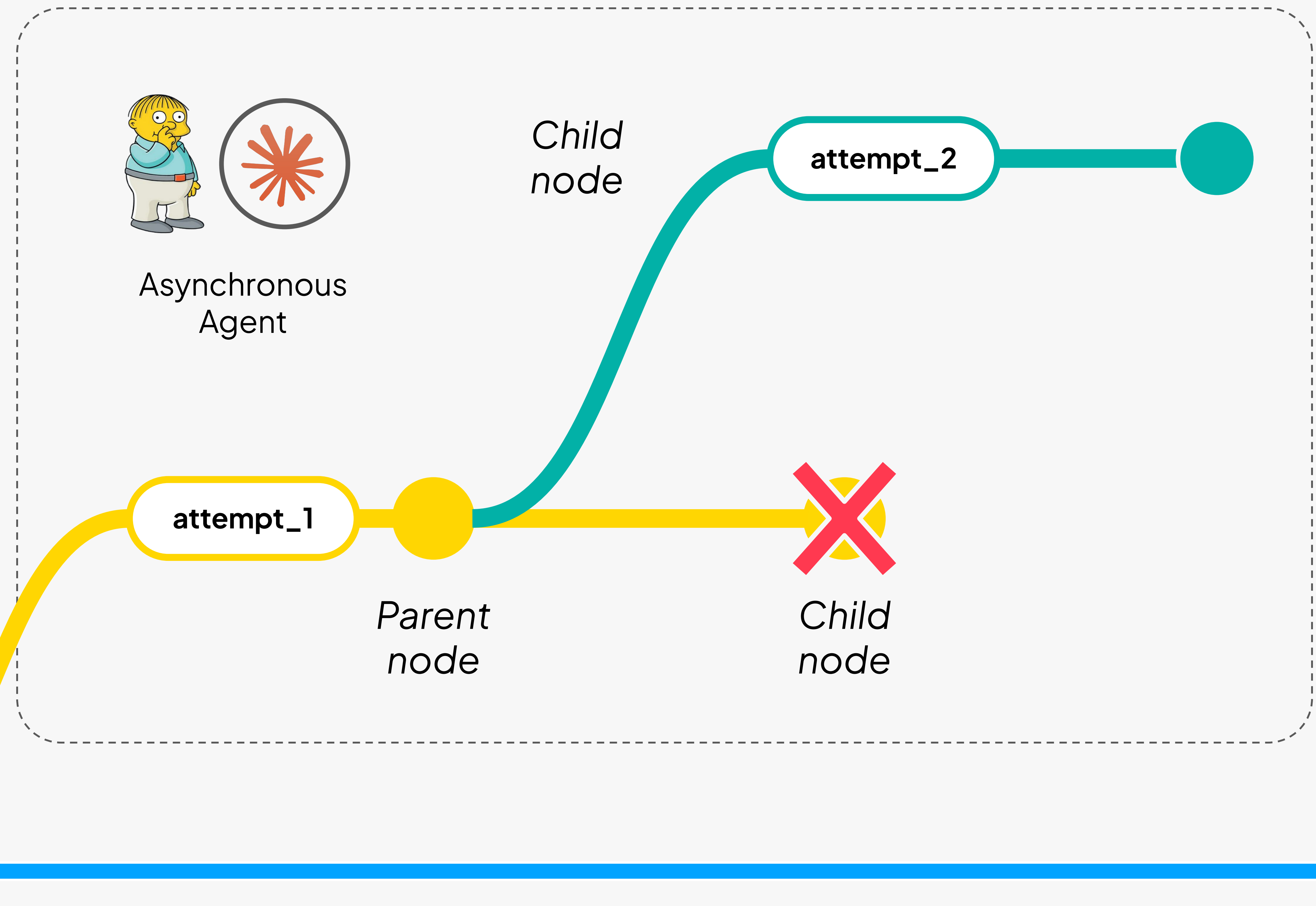


From merges to transactions: failure

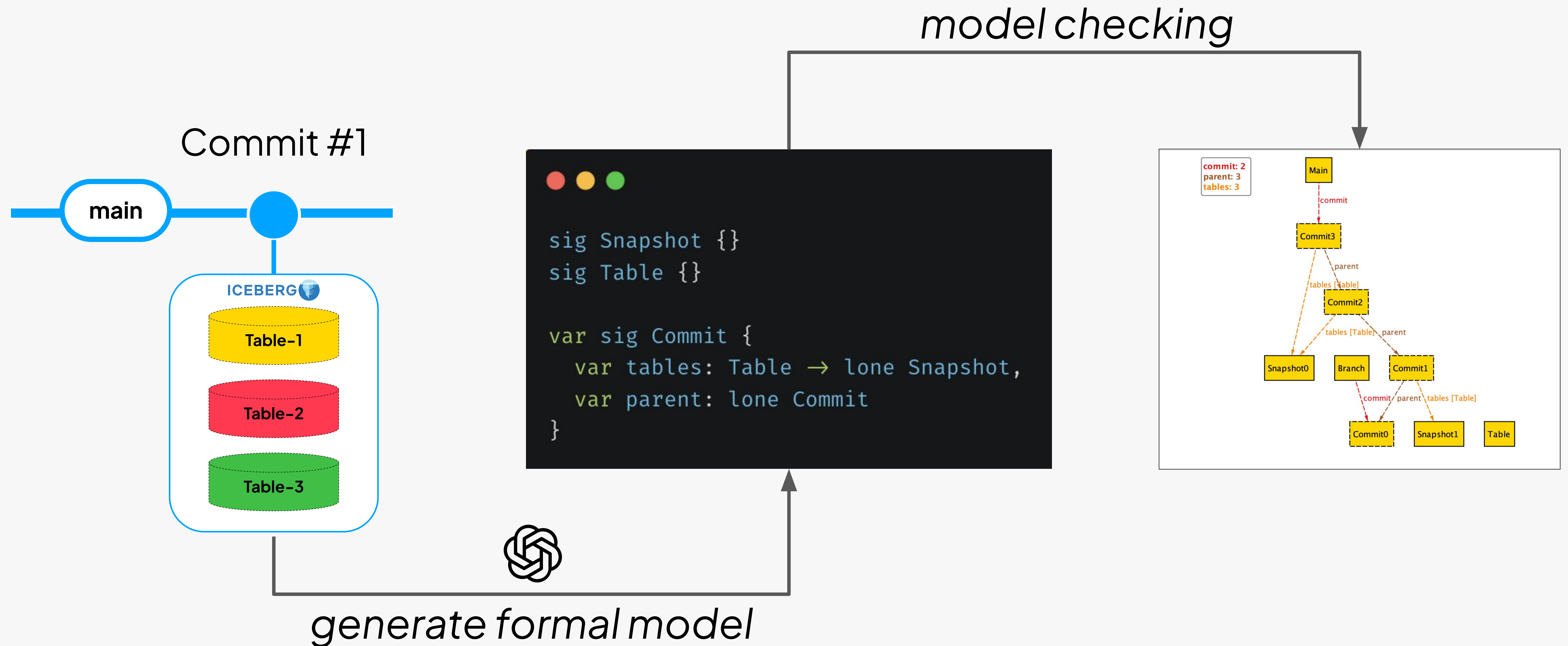


From transactions to durable execution

- **Durable execution:**
the functional nature of DAGs makes it easy to compose data assets by composing code.

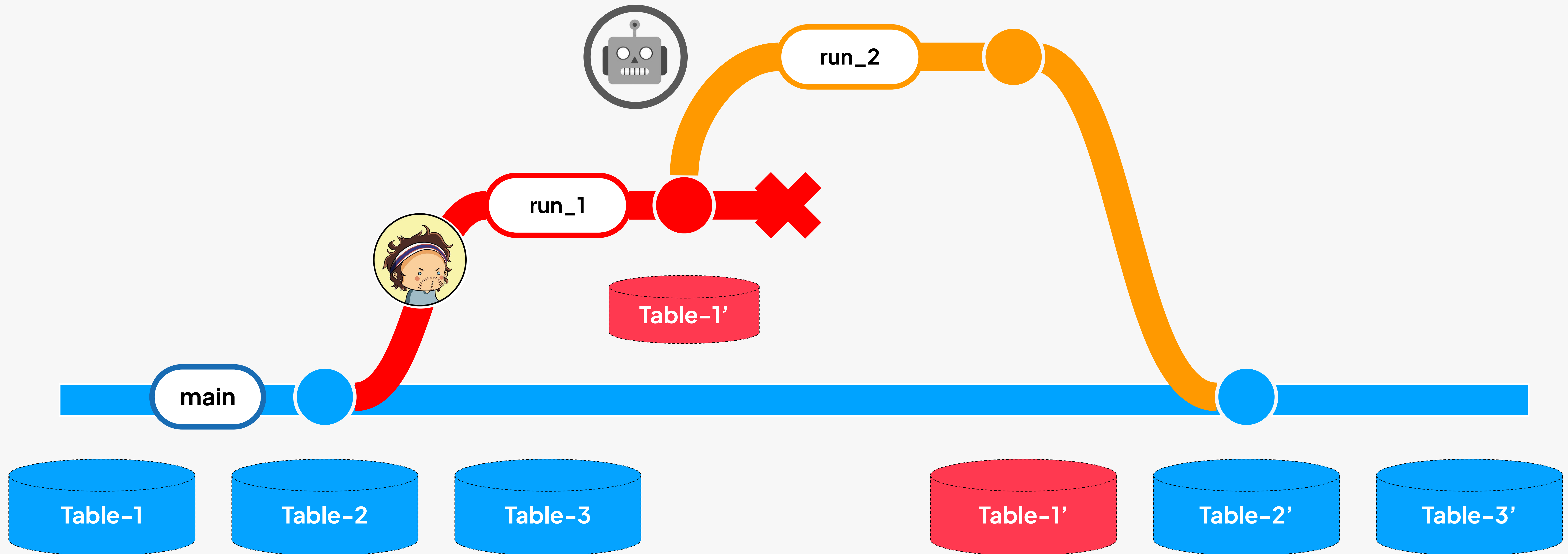


Is easy = correct? Taking correctness seriously!

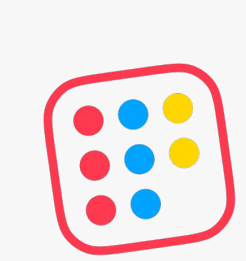


Million-to-one odds
happen eight times a
day in New York.

Can **main** ever become corrupted?

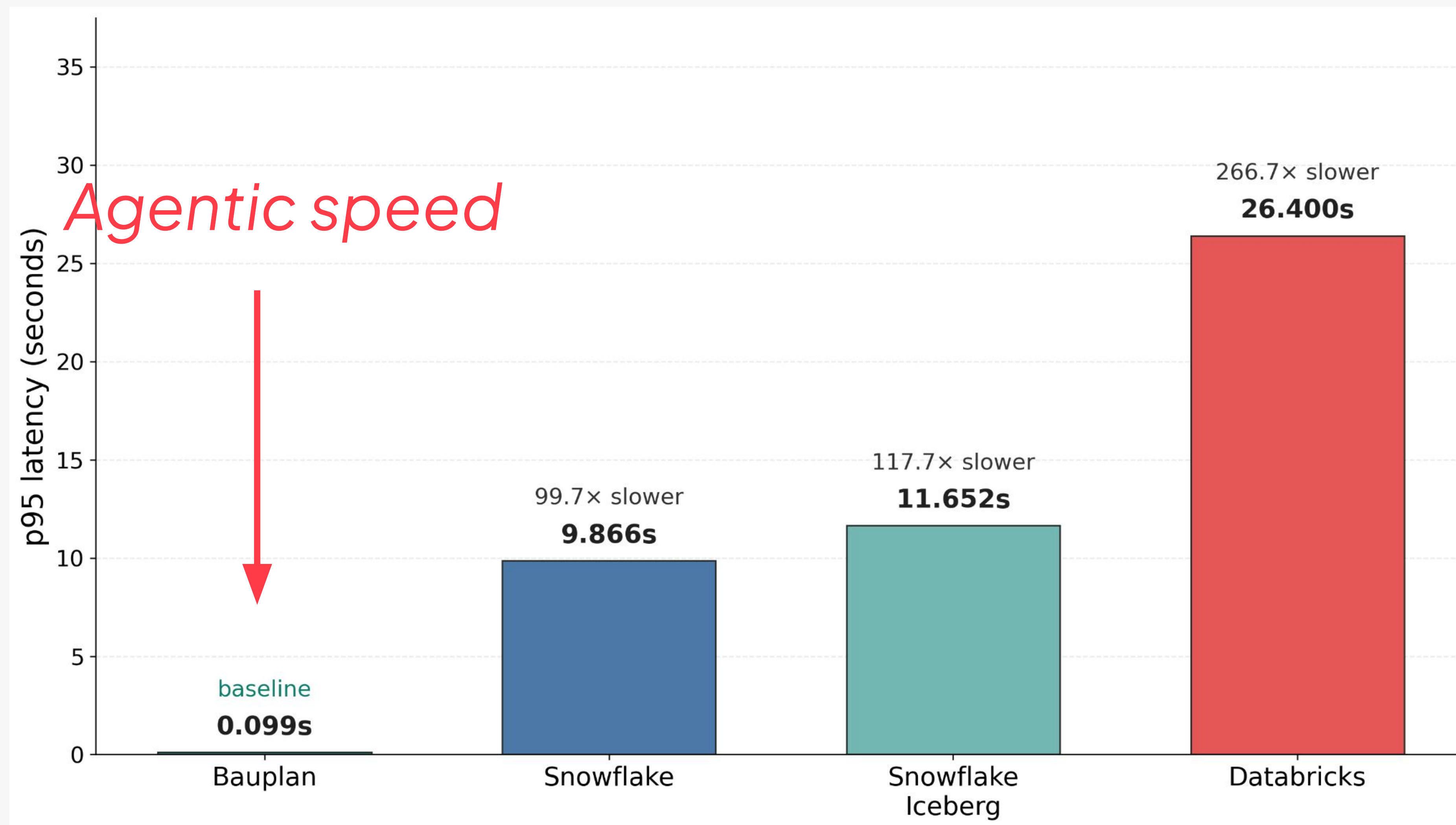


**In theory, there is no difference between
theory and practice. In practice...**



Does it scale?

- **Snowflake** pioneered OLAP “branching” with clones. But agentic speed is very different.



PlatformCustomersResources ▾Pricing

Agents Just Want To Have... Branches

The state of OLAP branching: Part 1

Jacopo TagliabueGiacomo Piccinini feat. Elaine Ang and Sam Weldon (Columbia University)

Jun 23, 2026

TL;DR: a Columbia University paper argues that branching databases are both crucial and far from optimal as coding agents become the primary users of OLTP systems. Under the same assumptions, we perform branching comparisons across lakehouses, highlighting the need for purpose-built systems such as Bauplan, which is ~100x faster than alternatives.

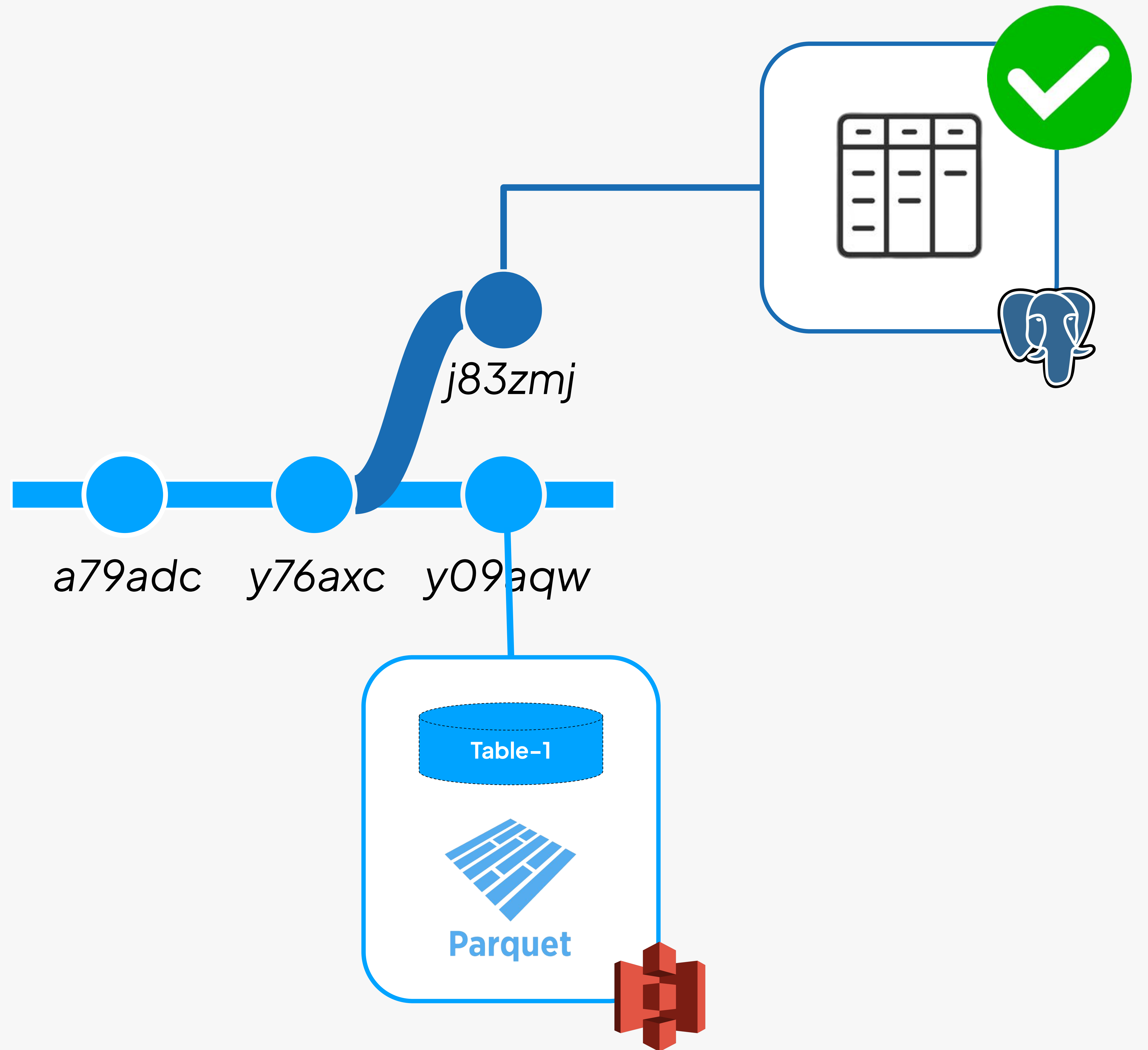
Last month, Columbia researchers presented a [new paper at ACM SAQ](#). “BranchBench” is a novel benchmark on OLTP databases for branching-first workloads, such as data cleaning and Monte Carlo simulations. In the era of coding agents and agentic data systems, how easily, quickly and cheaply can you branch your data, perform destructive operations (e.g. replace tables), query the results and decide whether to merge or discard the work?

Full benchmark [here](#)



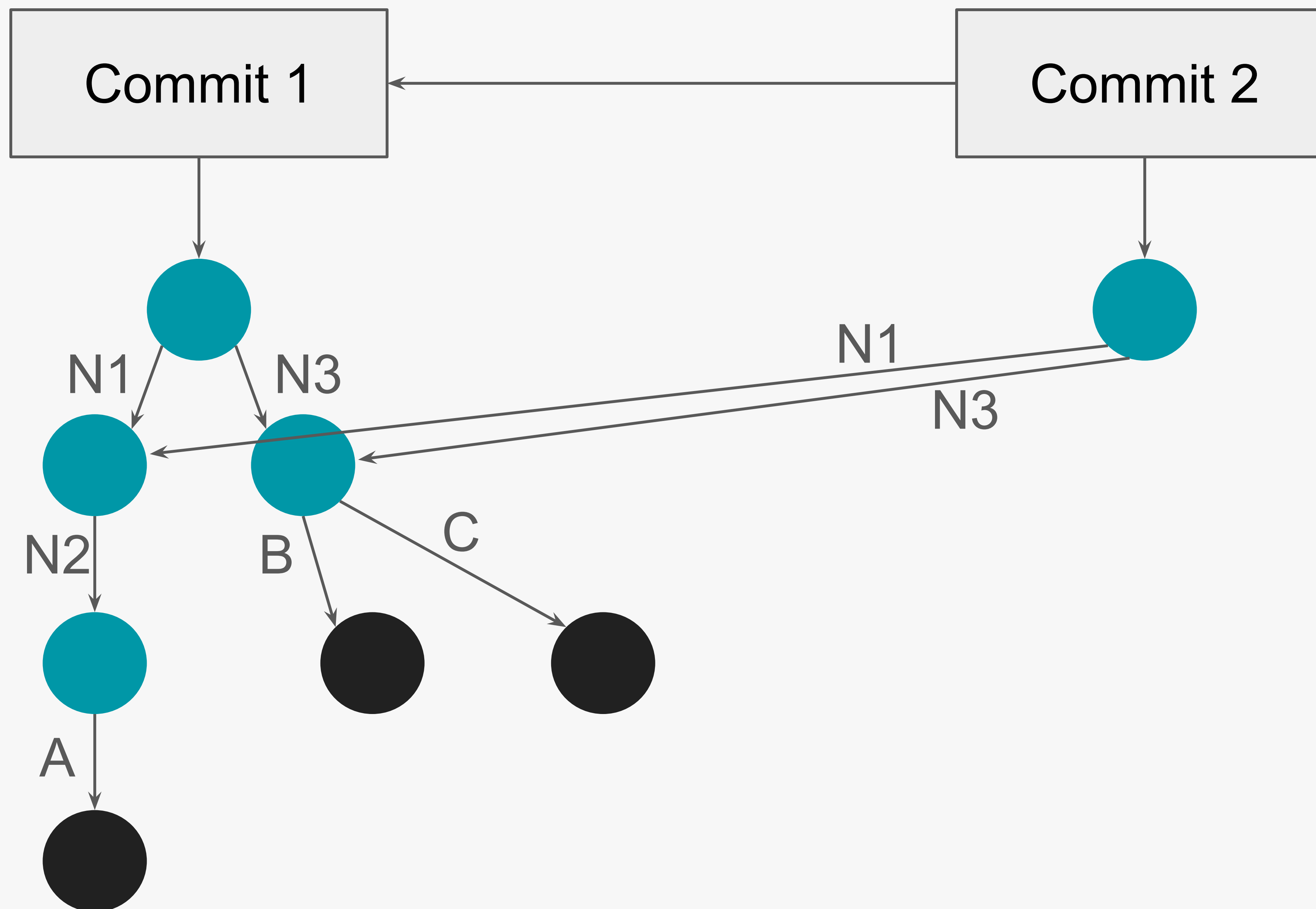
Does it scale?

- Branch creation and merging are metadata operations.
- Branching can easily scale to terabytes of data!
 - 1 new branch every 4 seconds
 - 60 ms to branch off
 - 80 ms to merge



Does it scale? Copy-on-write trees

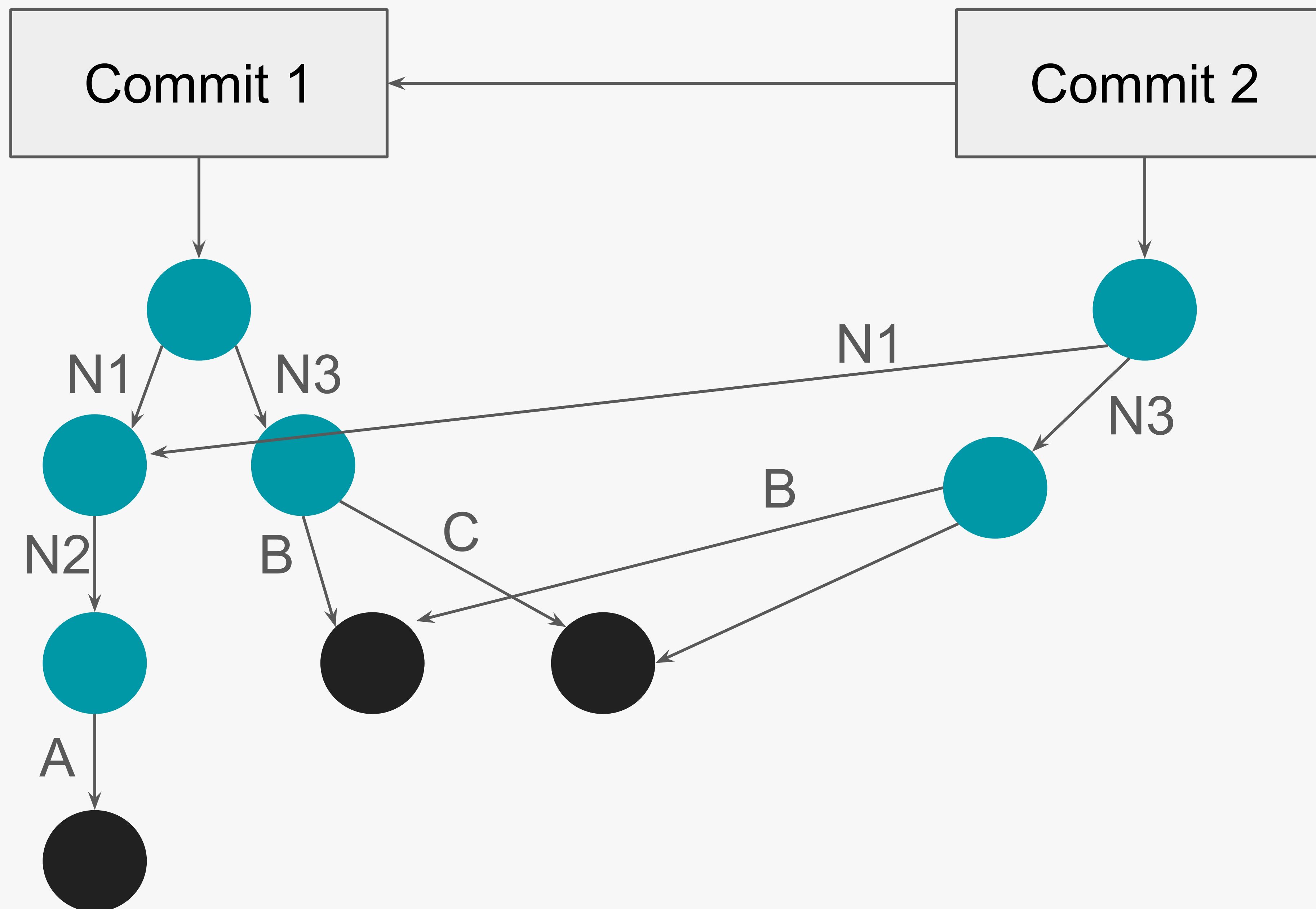
- What if I rename N3.C => N3.D?



1. Clone root

Does it scale? Copy-on-write trees

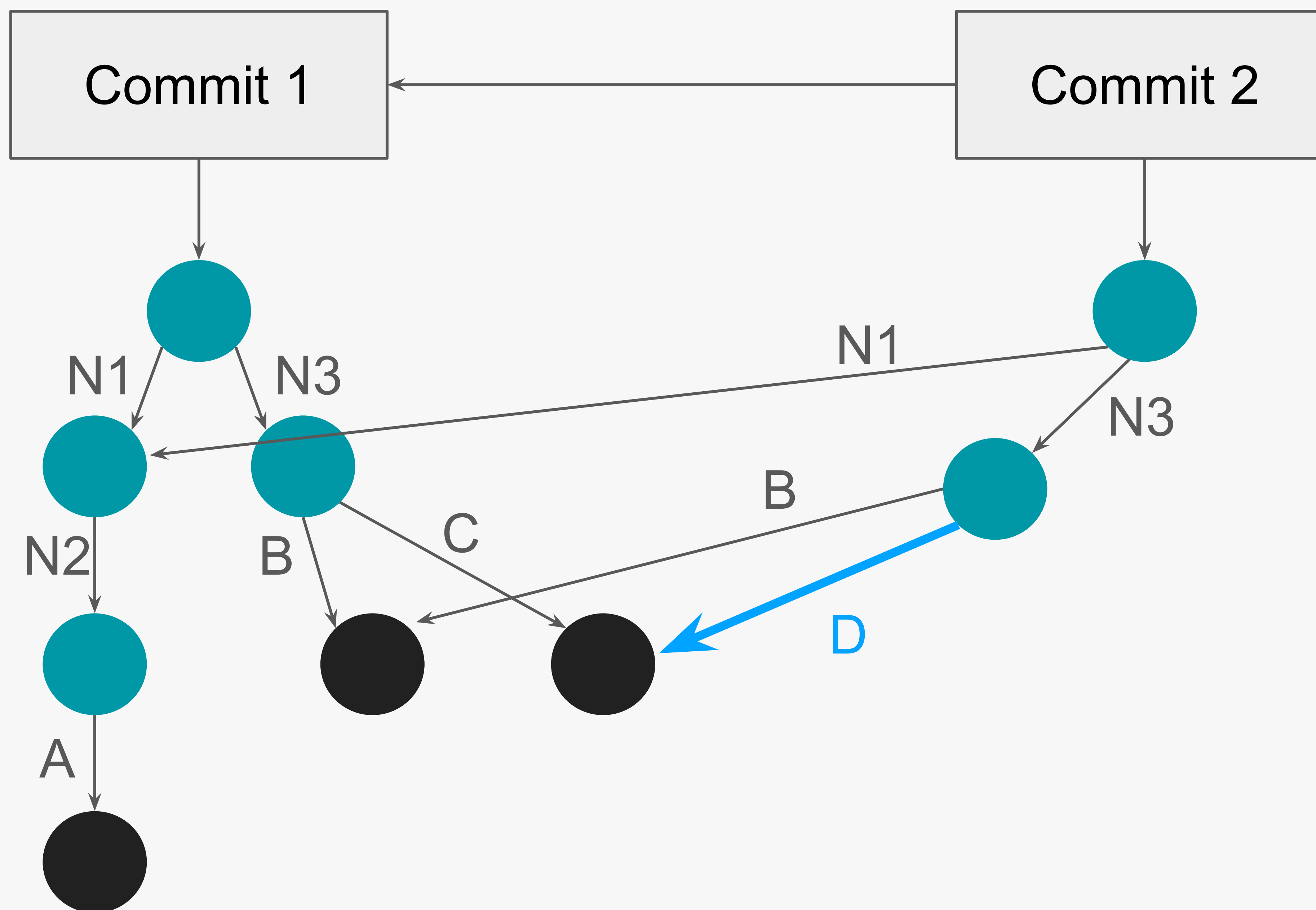
- What if I rename N3.C => N3.D?



1. Clone root
2. Clone modified namespace

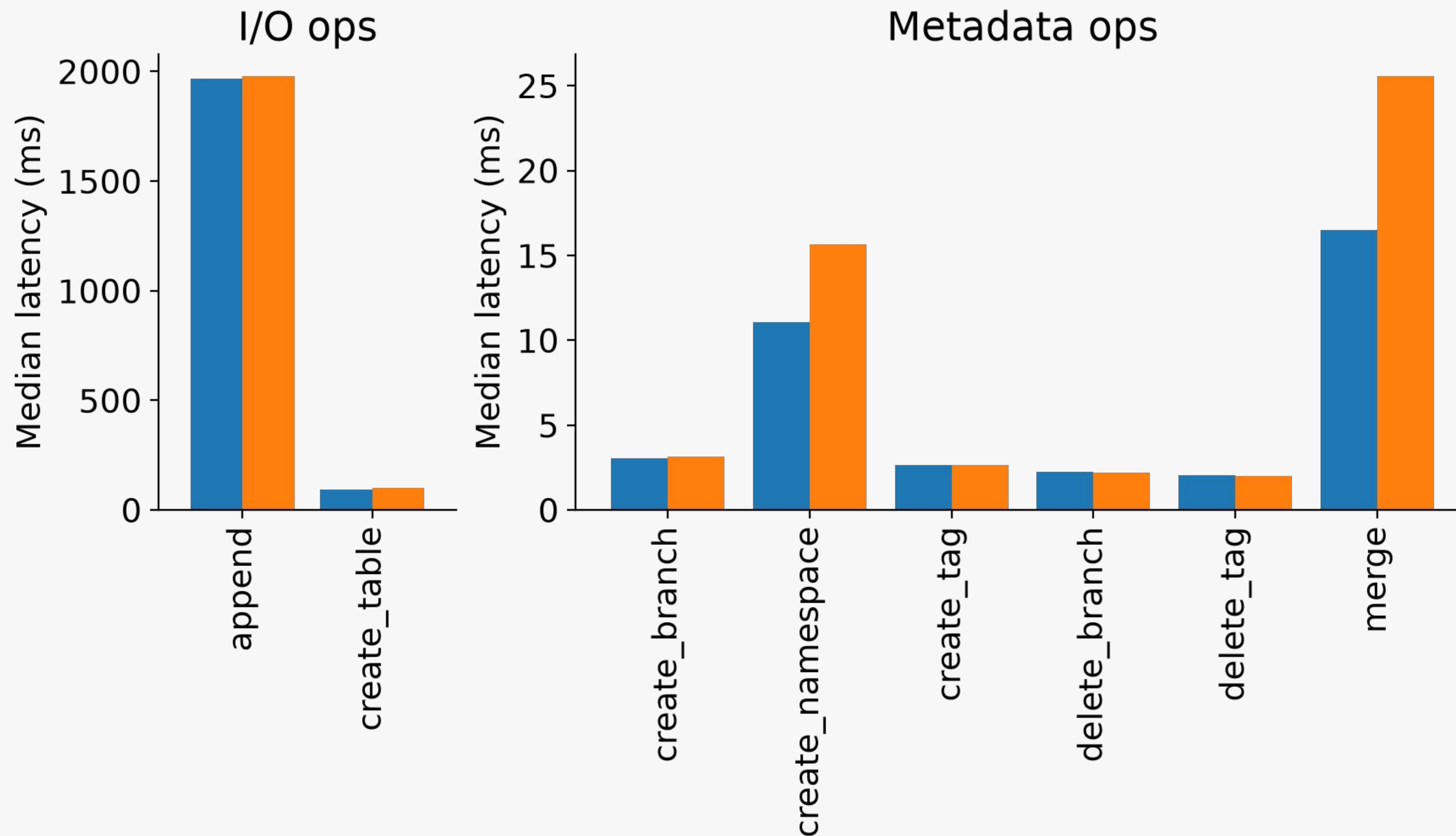
Does it scale? Copy-on-write trees

- What if I rename N3.C => N3.D?

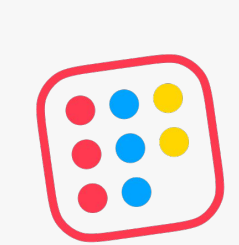


1. Clone root
2. Clone modified namespace
3. Apply update

Does it scale? Latency



- **Small (blue):** 32k commits, **128 tables**, 4 namespaces.
- **Big (orange):** 1 million commits, **1 million tables**, 1k namespaces.
- Data-intensive ops completely dominate, even at scale.



Conclusion

1. The road to the self-driving data stack goes through *Git-for-data*:
 - *concurrent writes, auditability, conflict management...*
2. We started from *scratch*: performance, correctness, and semantics are hard.
3. To complete the stack, we also need new APIs and sandboxed compute: **come and ask me about them!**

**WE DO THIS
NOT BECAUSE IT IS EASY,
BUT BECAUSE
WE THOUGHT
IT WOULD BE EASY.**

Building a Correct-by-Design Lakehouse

Data Contracts, Versioning, and Transactional Pipelines for Humans and Agents

Weiming Sheng*
Columbia University
USA

Jinlang Wang*
University of Wisconsin-Madison
USA

Manuel Barros*
Carnegie Mellon University
USA

Aldrin Montana*
Bauplan Labs
USA

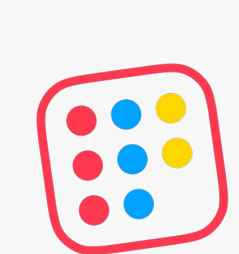
Jacopo Tagliabue*
Bauplan Labs
USA

Luca Bigon*
Bauplan Labs
USA

Abstract

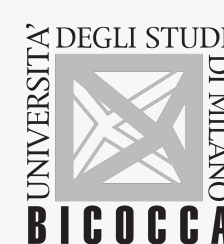
Lakehouses are now the default substrate for analytics and AI, but they remain fragile under concurrent, untrusted

as the majority of labor shifts from writing code to *verifying and approving changes*, correctness in the face of untrusted actors becomes non-negotiable for data workloads.



We are hiring!

Come work alongside the best data + AI minds in industry and academia!



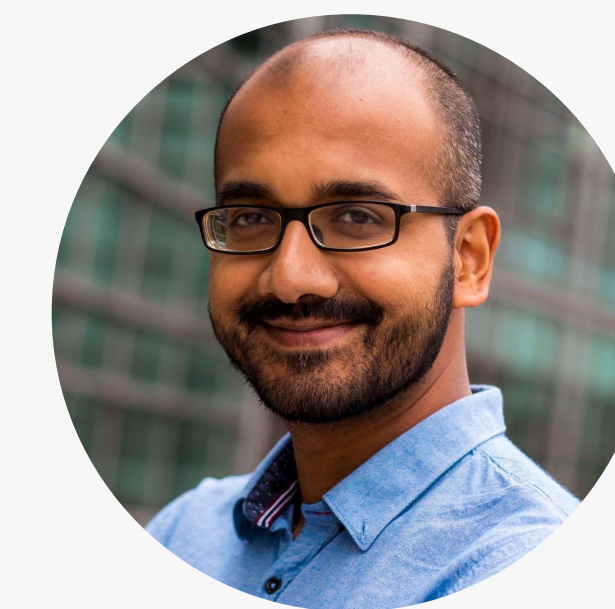
Wes McKinney
Creator of Arrow



Chris Re
Stanford AI Lab.



Ihab Ilyas
UWaterloo



Aditya Parameswaran
UC Berkeley



Paul Dix
InfluxDB



Eugene Wu
Columbia University



Remzi Arpaci-Dusseau
UW Madison

| jacopo.tagliabue@bauplanlabs.com

| Want to chat? **Reach out!**

“Skill issues”: data-centric optimization of lakehouse agents

Nicole Rose Schneider
nsch@umd.edu
University of Maryland
USA

Davide Ghilardi
davide.ghilardi@unimib.it
Università Milano Bicocca
Italy

GitLake: Git-for-data for the agentic lakehouse

Weiming Sheng*
Columbia University
USA

Jinlang Wang*
University of Wisconsin-Madison
USA

Manuel Barros*
Carnegie Mellon University
USA

ABSTRACT
Coding agents
success depend
round them. W
erating on a br
APIs and Git-L
code, branches
optimization p
candidate skills
both trace-level
state. The key
write-path agen
data changes

ABSTRACT

Test-Time Optimization of Physical Query Plans with LLMs

1 INTRODUCTION

Mehmet Hamza Erol
Stanford University

Xiangpeng Hao
University of Wisconsin-Madison

Federico Bianchi
TogetherAI

Backed by

